The SQL language
(Part 3)
DDL, DML, Views, Privileges

DDL
(Data Definition Language)

Schema Definition in SQL

Domains

- A domain specifies the valid values for an attribute
  - An integer is any integer number
  - “Color” is any of a list of (how many?) pre-set color-names
  - “Age” is an integer (> 0 with a max based on the species)
  - “Price” is a positive number with 2 decimal digits…
- Two categories:
  - **Elementary** (defined by the standard, also called “built-in”)
    - SQL-2 comes with 6 types
  - **User-defined:** as restrictions of built-in domains

Built-in Domains (1/6)

- Characters
  - Character or string;
  - Strings may have variable length;
  - May use a different character set (e.g.., Latin, Greek, Cyrillic, etc.)
  - Syntax:

    `character` [`varying`] [(*Length*)] [`character set` *Set*]
  - Alternatively, shorthands are defined: `char` and `varchar`, for `character` and `character varying`, respectively
  - Examples:
    - `char(6)` → `character(6)`
    - `varchar(50)` → `character varying(50)`

Built-in Domains (2/4)

- Exact numerical domains:
  - **Exact** values, integers, decimals… 4 alternatives:

```
numeric [ ( Precision [, Scale ] ) ]
decimal [ ( Precision [, Scale ] ) ]
integer
smallint
```

NUMERIC (5,2) → [-999.99 , +999.99]

NUMERIC and DECIMAL are synonyms in MySQL, Transact-SQL and Oracle

- Approximated numerical values:
  - floating point : internally represented as mantix + exponent

```
float [ ( Precision ) ]
real
double precision
```

The actual precision is implementation dependent

Built-in Domains (3/4)

- Timestamps
  - `date (year, month, day)`
  - `time[(` *Precision* `)][with time zone]:(hour, minute, second)`
  - `timestamp[(` *Precision*`)][with time zone]`

Oracle: `timestamp(4) with time zone`
`28-JAN-18 03.35.39.4636 PM +01:00`

- Time intervals

  `interval` *LargestTimeUnit* [`to` *SmallestTimeUnit*]

  - Two groups of time units:
    - { year, month } and { day, hour, minute, second }
  - Examples
    - `interval year to month`
    - `interval second`

MySQL

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(*fsp*)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(*fsp*)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(*fsp*)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000.

Built-in Domains (4/4)

- Bit
  - Boolean values (true/false), one bit, or a sequence of bits
  - Syntax: `bit [ varying ] [ (`*length*`) ]`
- New domains in SQL:1999
  - `BLOB` Binary Large Object
  - `CLOB` Character Large Object
- SQL:1999 also introduces aggregate constructors
  - `REF`, `ARRAY`, `ROW`…
  - beyond the pure relational data model !
    - not considered further here

User-defined Domains

- It is possible to define ad-hoc domains as restrictions of built-in domains. Each “user-defined” domain features:
  - A name
  - A “starting” elementary domain
  - A(n optional) default value
  - Some (optional) constraints
- Syntax:

  `create domain` *DomainName* `as` *ElementaryDomain* [*DefaultValue*] [*Constraints*]
- Example: `create domain Grade as smallint default 18`
- Useful to define domains that can be re-used in more than one table, and to make the schema more readable

Default

- Defines the value of an attribute in case no value is specified at tuple insertion time
- Syntax:

  `default` < *Value* | `user` | `null` >
- *Value* must be a value compatible with the domain
- `user` is the username of the logged user issuing the command

The “null” value

`null`

- A ***polymorphic*** value (i.e., belonging to any domain), that stands for an “unknown” value:
  - A value that applies, but not known to the database
    - e.g. birth date
  - A value that does not apply
    - e.g. driving license for under 18
  - A value that it is unknown whether applies or not:
    - e.g. driving license for over 18

Schema Definition

- A *schema* is a collection of objects:
  - domains, tables, indexes, assertions, views, privileges… (*not all covered here*)
- A schema has a name and an owner

Syntax:

```
create schema [ SchemaName ]
  [ [ authorization ] Authorized ]
  { SchemaElementDefinition }
```

Examples on real DBMSs

POSTGRESQL

```
CREATE SCHEMA schema_name
    CREATE TABLE table_name1 (...)
    CREATE TABLE table_name2 (...)
    CREATE VIEW view_name1
        SELECT select_list FROM table_name1;
```

MYSQL

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
    [create_specification] ...
```

Table Definition

- A SQL table consists of:
  - an ordered list of attributes
  - a set of constraints (possibly, empty)
- Command `create table`
  - defines the table schema and creates an empty instance
- Syntax:

```
create table TableName (
    AttributeName Domain [DefaultValue] [Constraints]
    { , AttributeName Domain [DefaultValue] [Constraints] }
    [OtherConstraints]
)
```

Example

```
create table Student (
    SId       character(6) primary key,
    Name       varchar(30) not null,
    City       varchar(20),
    Dept       char(3)
)

create table Exam (
     SId          char(6),
     CId          char(6),
     Date         date  not null,
     Grade        smallint not null,
     primary key(SId,CId)
)

create table Course (
     CourseId  char(6) primary key,
     Name       varchar(30) not null,
     Professor varchar(20)
)
```

Constraints

- Constraints control and specify the valid values for every instance of the database
- Constraints can be classified as
  - **Intra-relational** constraints (within one table)
  - Can be further distinguished as
    - tuple-level
    - table-level
  - **Inter-relational** constraints (between two tables)
    - Mainly to express referential integrity

Intra-relation Constraints

- `not null`
  - No null values for the attribute: *tuple level*
- `unique`:
  - no duplicates allowed (defines candidate keys): *table level*
  - syntax:
    - for one attribute, only: `unique`, after the domain;
    - for more attributes: `unique(` *Attribute* {`,` *Attribute* } `)`
- `primary key`
  - Defines the primary key (exactly one in each table)
    - implies `not null` and `unique` : *table level*
- `check`
  - Introduces an arbitrary condition
    - an expression that must always be true
    - example `check( not exists ( select … from … ))`

Intra-relation Constraints: Examples

- Every couple of attributes Name and Surname univocally identifies every tuple:

```
Name varchar(20) not null,
Surname varchar(20) not null,                (A)
unique(Name,Surname)
```

- **Not** the same as:

```
Name varchar(20) not null unique,
Surname varchar(20) not null unique          (B)
```

Which one is more restrictive? A or B ?

Referential Integrity

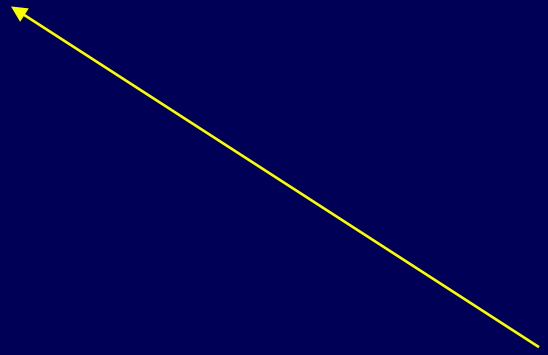
- `references` and `foreign key` for referential integrity:
  - for one attribute only:
    `references` after the domain
  - for several attributes:
    `foreign key (` *Attribute1* {`,` *Attribute2* } `)`
    `references` …
- The foreign key in the “internal” table must take values within those listed in the “referenced” columns of the “external” table
- Reaction policies can be defined to manage violations of the referential integrity

Example: Student - Exam

Student

SId	
123	
415	
702	

External table



Exam

SId	
123	
123	
702	

Internal table

Dangling references

Student

SId	
123	
415	
702	

External table

What if a student is deleted? what happens to the exams that were referring to that student?

Exam

SId	
123	
123	
702	

Internal table

Dangling references

- Reaction policies are applied to the “internal” table(s) after changes occur on the “external” table
- Violations can derive from (1) **updates** of the attribute under consideration OR (2) tuple **deletion**
- Reaction policies:
  - `cascade`: propagates the update
  - `set null`: forces to null the value of the attribute
  - `set default`: forces the attribute to the default value
  - `no action`: operation is blocked
- Reactions can depend on the occurring event. Syntax:

```
on < delete | update >
   < cascade | set null | set default | no action >
```

Reactions to `delete`

What happens to the exams when a student is **deleted**?

- **`cascade`**
  the exams are also deleted
- **`set null`**
  `Exam.SId` is set to **null** for the exams of the deleted student (**beware**: *inapplicable if part of the primary key*)
- **`set default`**
  the `Exam.SId` is set to the default value for the exams of the deleted student (*inapplicable if primary key*)
- **`no action`**
  the deletion of the student is forbidden (the delete command is aborted and has no effect)

Reactions to `update`

What happens to the exams when a student is **updated**?

- **`cascade`**

  SId in the Exam table is analogously updated in the exams of the updated student (*so that the reference is still valid*)

- **`set null`**

  SId in the Exam table is set to **null** for the exams of to the updated student (*inapplicable if primary key*)

- **`set default`**

  SId in the Exam table is set to the default value in the exams of the updated student (inapplicable if primary key)

- **`no action`**

  SId in Student can’t be updated (the update is rejected)

Referential integrity in the table definition

```
create table Exam
( ....
    primary key(SId,CourseId)
    foreign key SId
      references Student on delete cascade
                           on update cascade
    foreign key CId
      references Course(CourseID)
                       on delete no action
                       on update no action )
```

Alternative syntax (in the internal table)

```
create table Exam
   ( SId char(6),
     ...,
     foreign key Sid
       references Student( Sid )
        on delete cascade
        on update cascade )
```

```
create table Exam
   ( SId char(6) references Student( Sid )
        on delete cascade
        on update cascade,
        ... )
```

```
create table Exam
   ( SId char(6) references Student
        on delete cascade
        on update cascade,
        ... )
```

A wrong instance

Student

SId	Name	City	Dept
123			
415			
702			

Exam

SId	CourseId	Date	Grade
123	1	7-9-13	30
123	2	8-1-14	28
123	2	1-8-13	23
702	2	7-9-13	25
702	1	NULL	NULL
714	1	7-9-13	26

Primary key violation

not-null violation

referential integrity violation

A Correct Instance

Student

SId	Name	City	Dept
123			
415			
702			

Exam

SId	CourseId	Date	Grade
123	1	7-9-13	30
123	2	8-1-14	28
702	2	7-9-13	25

Another example: Management of Orders

```
create table Customer (
    CustId    char(6)  primary key,
    Address   char(50),
    SSN       char(12) unique
)

create table Product (
    ProductId char(6)  primary key,
    Name      char(20),
    Price     smallint
)
```

Order

```
create table Order (
   OrderId   char(6)  primary key,
   CustId    char(6)  not null default='999999’,
   Date      date,
   Amount    integer,
   foreign key CustId
     references Customer
         on delete set default
         on update set default
)
```

Line

```
create table Line (
   OrderId  char(6),
   ProductId char(6),
   Qty      smallint,
   primary key(OrderId, ProductId)
   foreign key OrderId references Order(OrderId)
     on delete cascade on update cascade
   foreign key ProductId
     references Product(ProductId)
        on delete no action
        on update no action
)
```

Creation of Indexes

An index is a data structure that facilitates efficient access to data according to some search criterion

create index

```
create index DataIx on Order(Date)
```

create unique index

```
create unique index OrdKey on Order(OrderId)
```

Physical access structures and indexing are not covered in detail in this course (→ Databases 2)

DML
(Data Manipulation Language)

Commands to change the ***schema*** and the ***instance*** of a database

Commands to change the **Schema** of a DB

- The Schema of a database may need to be maintained, in case new requirements arise
  - Need to add(remove) columns to(from) an existing table
  - Modification of existing table with more(fewer) constraints
- SQL commands:
  - **`alter`** `(alter domain … , alter table … )`
    to modify existing objects
  - **`drop`**
    `drop <schema | domain | table | view | assertion>`
    ComponentName `[ restrict | cascade ]`
    to remove existing parts of the schema (domains, views, constraints, …)

Modification of existing components

- **alter**
  - Applies to domains and tables. Examples:

```
alter table Order
      add column InvoiceNumber char(6)
      add column NumPieces integer > 0

alter table Order
    alter column Amount
        set default 0

alter table Order
    drop column Date
```

Removal of existing components

- **`drop`**
  - for domains, tables, indexes, views, assertions, procedures, triggers
    - **e.g.**: `drop table Order`
    - **e.g.:** `drop index DataIx`

- Options `restrict` and `cascade`
  - **`restrict`**
    - aborts the `drop` command if objects include any instance
    - only empty objects can be removed
  - **`cascade`**
    - extends the `drop` command to also remove linked objects (tuples in other tables with referential integrity)
    - The cascading effect can be catastrophic!

Relational Catalogue (and data dictionary)

- The relational catalogue is an internal description of the data structures within a database
  - The description is expressed in the relational model
- Every system has the catalogue organized in its own way
- According to the SQL-2 standard it is organized onto 2 levels
  - `Definition_Schema`
    - tables that contain the description of all the data structures within the database
  - `Information_Schema`
    - views defined onto the Definition_Schema

Information Schema

SQL-2 prescribes 23 views, among which:

– Domains
– Domain_Constraints
– Tables
– Views
– Columns
– ...

It gives a centralized description of the schema:

- Table names
- Attribute names and size,
- number of records in each table
- Constraints
  - (such as the range of permitted values)
- Detailed information about the physical database
  - storage, access paths, files, and record length
- Access authorization
  - permissions and access rights
- Usage statistics

```
mysql> SHOW TABLES FROM INFORMATION_SCHEMA;
+---------------------------------------+
| Tables_in_INFORMATION_SCHEMA          |
+---------------------------------------+
| CHARACTER_SETS                        |
| COLLATIONS                            |
| COLLATION_CHARACTER_SET_APPLICABILITY |
| COLUMNS                               |
| COLUMN_PRIVILEGES                     |
| ENGINES                               |
| EVENTS                                |
| FILES                                 |
| GLOBAL_STATUS                         |
| GLOBAL_VARIABLES                      |
| KEY_COLUMN_USAGE                      |
| PARTITIONS                            |
| PLUGINS                               |
| PROCESSLIST                           |
| REFERENTIAL_CONSTRAINTS               |
| ROUTINES                              |
| SCHEMATA                              |
| SCHEMA_PRIVILEGES                     |
| SESSION_STATUS                        |
| SESSION_VARIABLES                     |
| STATISTICS                            |
| TABLES                                |
| TABLE_CONSTRAINTS                     |
| TABLE_PRIVILEGES                      |
| TRIGGERS                              |
| USER_PRIVILEGES                       |
| VIEWS                                 |
+---------------------------------------+
```

The **Columns** view

- Columns can have many attributes:
  - Table_Name
  - Column_Name
  - Ordinal_Position
  - Column_Default
  - Is_Nullable
  - …

  …

  … *and many more*

Columns

Table_Name	Column_Name	Ordinal_ Position	Column_ Default	Is_Nullable
Impiegato	Matricola	1	null	N
Impiegato	Cognome	2	null	N
Impiegato	Nome	3	null	N
Impiegato	Dipart	4	null	Y
Impiegato	Ufficio	5	null	Y
Impiegato	Stipendio	6	0	Y
Dipartimento	Nome	1	null	N
Dipartimento	Indirizzo	2	null	Y
Dipartimento	Città	3	null	Y

TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE	CHARACTER_MAXIMUM_LENGTH
nucleotide_variant	start_alternative	6	NULL	YES	int	NULL
nucleotide_variant	variant_length	7	NULL	NO	int	NULL
nucleotide_variant	variant_type	8	NULL	NO	text	65535
sequence	sequence_id	1	NULL	NO	int	NULL
sequence	experiment_type_id	2	NULL	NO	int	NULL
sequence	virus_id	3	NULL	NO	int	NULL

Reflexivity of the Catalogue

- All the data structures of the Catalogue are described within the Catalogue

Columns

Table_Name	Column_Name	Ordinal_ Position	Column_ Default	Is_Nullable
Columns	Table_Name	1	null	N
Columns	Column_Name	2	null	N
Columns	Ordinal_Position	3	null	N
Columns	Column_Default	4	null	Y
Columns	Is_Nullable	5	Y	N

- The DDL is a quick, reliable, portable, consistent and abstract tool for defining the relational data structures
- Each DDL command, however, is implemented via suitable DDML commands that modify the catalogue
- The catalogue is automatically maintained, and **shall never be directly modified** by the users
- **This approach facilitates portability within heterogeneous systems**

Commands to change the **Instance** of a DB

- Commands for:
  - Inserting new tuples (**`insert`**)
  - Removing existing tuples (**`delete`**)
  - modifying values of attributes (**`update`**)

- All the instructions work over sets (set-oriented)

- The command may include a condition, where any external table(s) can be referenced

Insert

- Syntax:

  insert into *Table* [(*AttributeList*)]
  < values (*ValueList*) | *SelectSQL* >

- Examples:

```
insert into Student
values ('456878', 'Giorgio Rossi',
        'Bologna', 'Logistics')

insert into Bolognesi
 ( select *  from Student
   where City = 'Bologna' )
```

Insert

- The sequence according to which attributes and values are cited is **relevant** (positional notation: the first value refers to the first attribute, and so on)

- If *AttributeList* is omitted, SQL refers to all the attributes of the relation, in the sequence they appear in the `create table` statement

- If *AttributeList* does not include all the attributes of the relation, the remainder attributes will assume the *default* value (or, if not defined, the null value)

Insert

- By means of an *AttributeList*:

```
insert into Student(SId,Name,City,Dept)
    values('456878', 'Giorgio Rossi',
           'Bologna', 'Logistics')
```

- By means of a query with an equivalent *AttributeList*:

```
insert into Bolognesi(Id,Name,City,Dept)
    ( select SId, Name, City, Dept
      from Student
      where City = 'Bologna' )
```

Delete

- Syntax:

  `delete from` *Table* [`where` *Condition*]

- Remove the student whose id is 678678:

```
delete from Student
 where SId = '678678'
```

- Remove the students who never passed an exam:

```
delete from Student
 where SId not in ( select SId
                    from Exam )
```

Delete

- **`delete`** removes all the tuples that fulfill the given condition
- If the **`where`** clause is omitted, then **all** the tuples are removed
- The command may generate removal of tuples in other tables, if referential integrity is enabled with a **`cascade`** policy
- To remove all the tuples of `Student` (keeping the empty schema of the table):

```
delete from Student
```

- To completely remove the table (including its schema):

```
drop table Student cascade
```

Update

- Syntax:

update *Table*
  set *Attribute* = < *Expression* | *SelectSQL* | null | default >
    {, *Attribute* = < *Expression* | *SelectSQL* | null | default >}
  [where *Condition*]

- Example:

```
update Exam
  set Grade = 30
    where Date = '1-4-13'

update Exam
  set Grade = Grade + 1
    where SId = '787989'
```

Update

- As the language is **set-oriented,** the order according to which we issue the commands is very relevant:

```
update Employee
  set Salary = Salary * 1.1
   where Salary <= 30 ;

update Employee
  set Salary = Salary * 1.15
   where Salary > 30 ;
```

- If commands are issued according to the order above, some lucky employees may receive a double increase

Selection conditions can be arbitrary

Increase of 5 euro the amount of all the orders which include the product ‘456’

```
update Order
set Amount = Amount + 5
where OrderId in
   ( select OrderId
     from Line
     where ProductId = ’456’ )
```

DML and Use of Nested Queries

Assign to the (newly added) attribute NumPieces the sum of the quantities of the lines of an order

```
update Order O
  set NumPieces =
   ( select sum(Qty)
     from Line L
     where L.OrderId = O.OrderId )
```

Access Control

Privacy in a DBMS

- Privacy: protection of the database so as to control access to sensitive data and to avoid unauthorized modifications or disclosure

- **Authentication** (user identification with *username* and *password*):
  - When connecting to the DBMS
  - When performing critical operations

```
CREATE USER name [ [ WITH ] option [ ... ] ]

option :    SYSID uid
          | CREATEDB | NOCREATEDB | CREATEUSER | NOCREATEUSER
          | IN GROUP groupname [, ...]
          | [ ENCRYPTED | UNENCRYPTED ] PASSWORD 'password'
          | VALID UNTIL 'abstime'
```

syntax of PostgreSQL

- **Authorization** of Individual users and user groups
  - to read specific portions of each schema in the DBMS
  - to perform specific operations
  - to limit or extend the authorizations (***privileges***) of other users

Authorization (privileges)

- Every component of a schema can be protected (tabels, attributes, views, domains, ...)
- The creator of a resource (**the owner**) **assigns privileges** (authorizations) to the other existing users
- A pre-defined user **`_system`** (or **`root`**) represents the db admin and has full access to all resources
- A **privilege** is characterized by:
  - the target resource
  - the action that the privilege allows on the target resource
  - the user that assigns the privilege
  - the user that receives the privilege
  - the right of the receiving user to further extend the privilege to other users

Types of privileges

- The SQL standard lists several distinct types of privileges for each resource
  - **`insert`**: to insert a new item
  - **`update`**: to modify an item
  - **`delete`**: to remove an item
  - **`select`**: to access the resource with queries
  - **`references`**: to enable referential integrity constraints that involve the resource (as this affects its modifiability)
  - **`usage`**: to use the resource in a schema definition (i.e., applicable to a domain)

… implemented DBMSs offer many more specific/proprietary privileges

- **`index`**, **`trigger`**, **`alter`**, **`drop`**, **`truncate`**, **`maintain`**, **`execute`**, …

- **`all privileges`** is a shortcut that implies all defined privilege types

grant and revoke

- To grant a privilege:

  grant < *Privileges* | all [privileges] > on *Resource*
  to *Users* [with grant option]

  – the grant option specifies if the receiving user has the right to propagate the privilege to other users

- To revoke a privilege :

  revoke *Privileges* on *Resource* from *Users*
  [restrict | cascade]

  – if restrict the privilege is denied only to the specified users

  – if cascade it is also revoked to other users who had it granted by the specified users (cascadedly along the chain of trust)

Views

Views

- Views provide users with a virtual vision of tables
- Views are classified as:
  - simple views (select and project over **one** table, only)
  - complex views (with joins or aggregates)
- Syntax:

  `create view` *ViewName* [(*AttributeList*)] `as` *SelectSQL*

  [`with` [`local` | `cascaded`] `check option`]

  - *The* `check option` *is used to constraint any modification to the "underlying" tables: a modification is accepted only if it doesn't cause a change in the content of the view.*

Views

- Views in SQL-2 may refer to previously defined views, but no mutual dependency is allowed (recursion has been introduced iny SQL:1999)
- Views can be used to write complex queries:
  - view decompose the problem in a more readable solution
- Views are sometimes necessary to express some queries:
  - queries that combine aggregations over other aggregations
  - queries that use in a complex way the **union** operator

A Simple View

Order with the amount over 10.000 €

```
create view MainOrder as
   select *
   from Order
   where Amount > 10000
```



Order

1	3	1-6-17	50.000,00
4	1	1-7-17	12.000,00
6	3	3-9-17	27.000,00

View Composition

- View:

```
create view MainOrder as
    select *
    from Order
    where Amount > 10000
```

- Query:

```
select CustId
from MainOrder
```

- View composition:

```
select CustId
from Order
where Amount > 10000
```

Cascaded Simple Views

```
create view AdminEmployee
                    (Id,Name,Surname, Salary) as
     select Id, Name, Surname, Salary
     from Employee
     where Dept = 'Admin' and Salary > 10

  create view AdminJuniorEmployee as
     select *
     from AdminEmployee
     where Salary < 50
```

Views and Queries

- Find the customer who purchased for the largest total amount (without using views):

```
select CustId
from Order
group by CustId
having sum(Amount) >= all
       ( select sum(Amount)
         from Order
         group by CustId )
```

Views and Queries

- Find the customer who purchased for the greatest total amount (using views):

```
create view CustAmount(CustId,TotAmt) as
    select CustId, sum(Amount)
    from Order
    group by CustId

    select CustId
    from CustAmount
    where TotAmt = ( select max(TotAmt)
                     from CustAmount )
```

Views and Queries

- Find the average number of orders for every customer:
  - wrong query (SQL does not support nested aggregates):

```
select avg(count(*))
from Order
group by CustId
```

  - correct query (by means of a view):

```
create view CustOrd(CustId,NumberOrders) as
     select CustId, count(*)
     from Order
     group by Custid

select avg(NumberOrders)
from CustOrd
```

Views to modify data

- **View:**

```
create view MainOder as
 select *
 from Order
 where Amount > 10000
```

- **Modification:**

```
update MainOder
   set Amount = Amount * 1.05
   where CustId = '45'
```

- **Composition of the view definition and the statement:**

```
update MainOder
   set Amount = Amount * 1.05
   where CodCli = '45' and Amount > 10000
```

- This is only possible with **simple views** (**basic selection and projection**).
  - with more complex views (including join, aggregates, …) it may not be possible to trace the modifications back to the source data items